

Raft : algorithme de consensus

Qu'est-ce que Raft ?

Raft est un algorithme de consensus conçu pour permettre à un cluster de serveurs de se mettre d'accord sur une même suite d'opérations (un log répliqué).

Objectif principal : garantir un état cohérent et fiable malgré les pannes de certains nœuds du système.

Raft simplifie la compréhension du consensus distribué en séparant clairement les responsabilités : élection du leader, réplication du log et sécurité des données.



Pourquoi un algorithme de consensus ?

● Systèmes distribués

Dans un système distribué, les nœuds doivent s'accorder malgré les pannes et les retards réseau. Sans mécanisme de consensus, les données peuvent diverger et créer des incohérences critiques.

● Cohérence des données

Exemple : maintenir des métadonnées, configurations ou verrous cohérents entre plusieurs serveurs. La tolérance aux pannes est essentielle pour garantir la disponibilité du service.

Les rôles dans Raft

● Leader

Coordonne les opérations et reçoit les requêtes des clients. Seul nœud à écrire dans le log et à communiquer avec les followers.

● Follower

Réplique passivement les entrées envoyées par le leader. Répond aux requêtes AppendEntries et redirige les clients vers le leader.

● Candidate

État temporaire lors d'une élection. Demande les votes des autres nœuds pour devenir le nouveau leader du cluster.

Election du leader

Timeout aléatoire

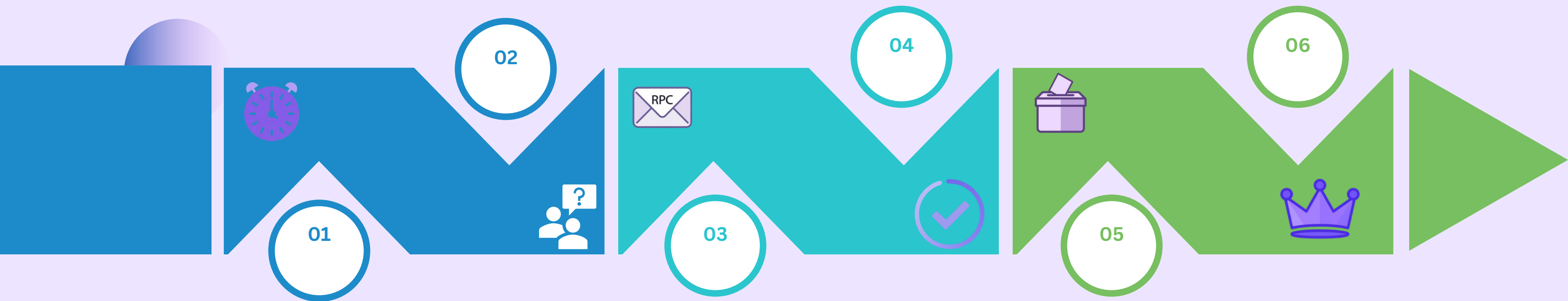
Le follower détecte l'absence de heartbeat après un délai aléatoire (150-300ms).

RequestVote RPC

Envoi de RequestVote(term, candidateId, lastLogIndex, lastLogTerm) aux autres nœuds.

Majorité [N/2]

Le candidat obtient la majorité des votes : $[N/2]+1$ sur N nœuds.



Passage Candidate

Le nœud incrémente son terme, vote pour lui-même et passe en état Candidate.

Réception votes

Les followers accordent leur vote si le terme est valide et le log à jour.

Devenir Leader

Le candidat devient Leader et envoie des heartbeats AppendEntries vides.

Réplication du log

AppendEntries RPC

Le leader envoie
AppendEntries RPC avec les
nouvelles entrées aux
followers.

Machine à États

L'état cohérent est
maintenu sur tous les
nœuds du cluster Raft.

Confirmation Followers

Les followers répliquent
l'entrée et envoient une
confirmation au leader.

Réception Commande

Le leader reçoit les requêtes
clients et les ajoute à son log
local.

Application État

Après commit, l'entrée est
appliquée à la machine à
états répliquée.

Commit Majoritaire

L'entrée est committée quand
 $\lceil N/2 \rceil$ nœuds l'ont répliquée
(majorité).



Sécurité et garanties

● Entrées committées

Un leader ne perd jamais d'entrées déjà committées. Une fois qu'une entrée est répliquée sur une majorité de nœuds et marquée comme committée, elle est garantie de persister même en cas de panne du leader actuel.

● Contraintes d'élection

Pour devenir leader, un candidat doit avoir un log au moins aussi à jour que la majorité des nœuds. Cette contrainte garantit que le nouveau leader possède toutes les entrées committées, assurant ainsi la cohérence du système.

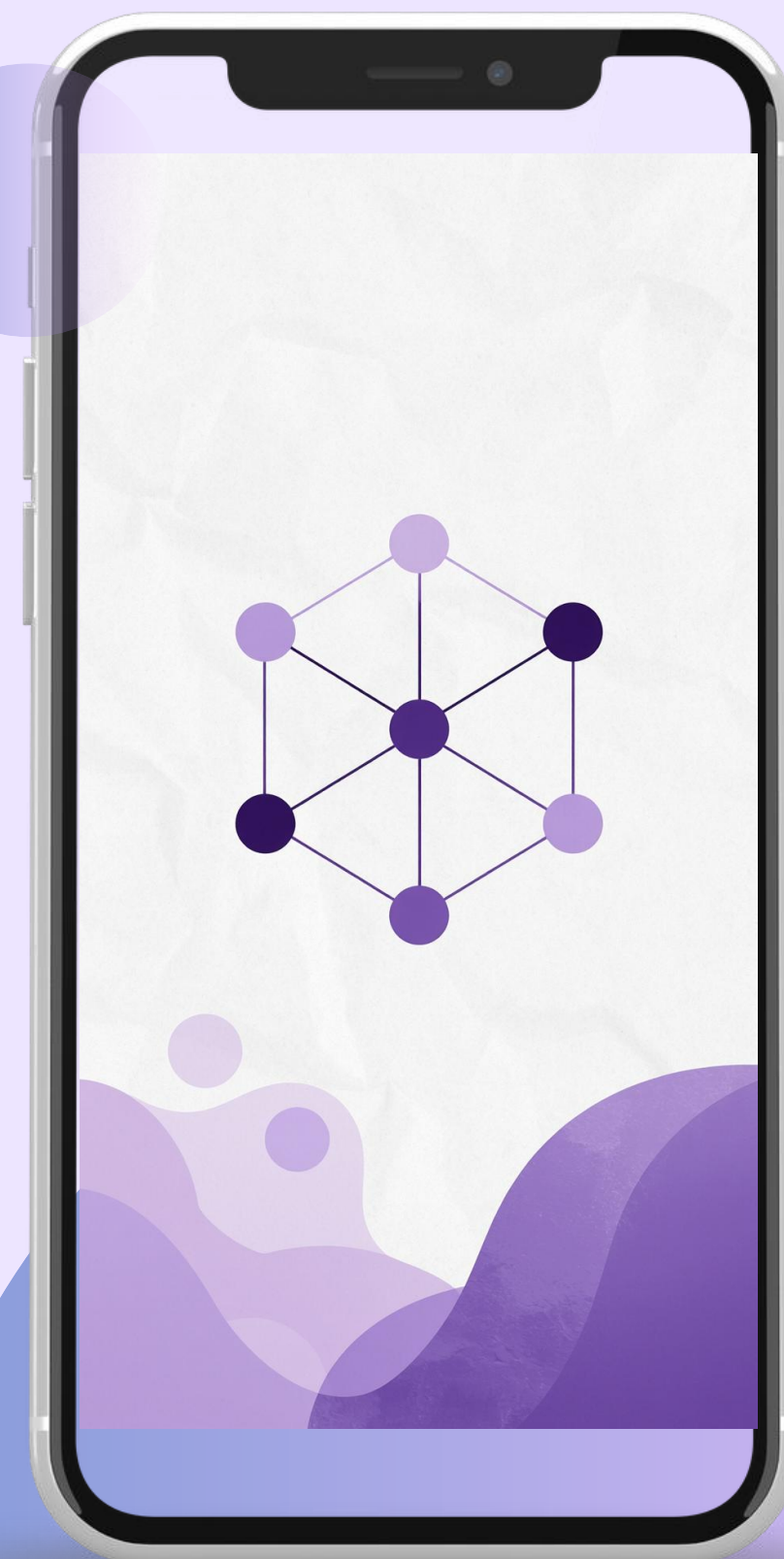
Changement de configuration

● Joint Consensus

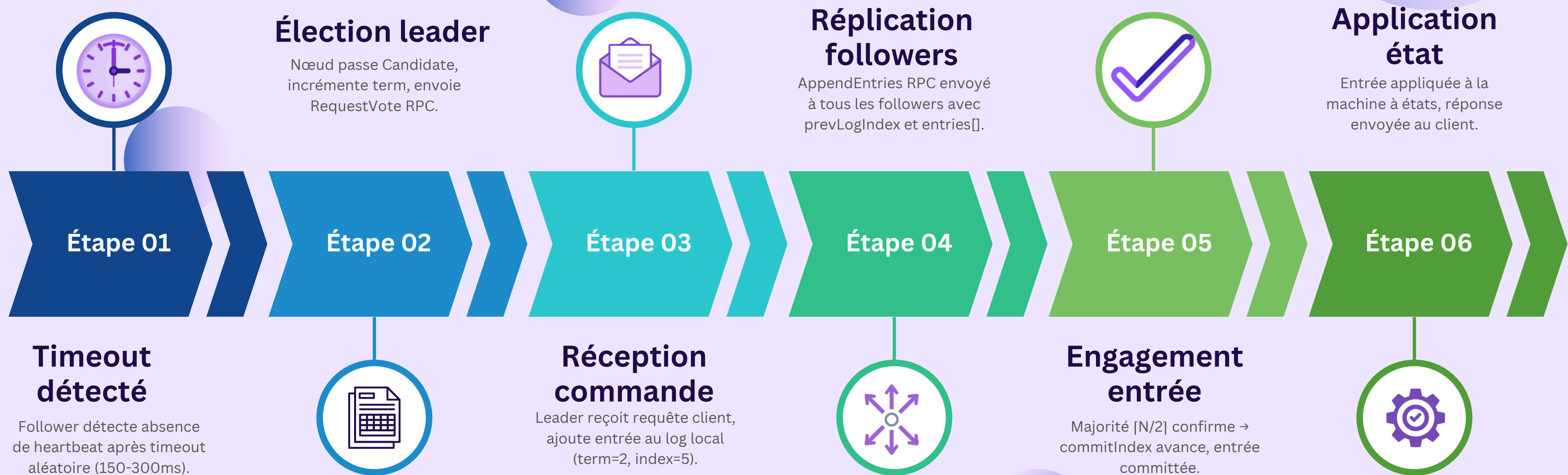
L'ajout ou le retrait de nœuds dans un cluster Raft s'effectue via un mécanisme de consensus conjoint ("joint consensus"). Cette approche permet de modifier la configuration sans interrompre le service ni risquer d'incohérences.

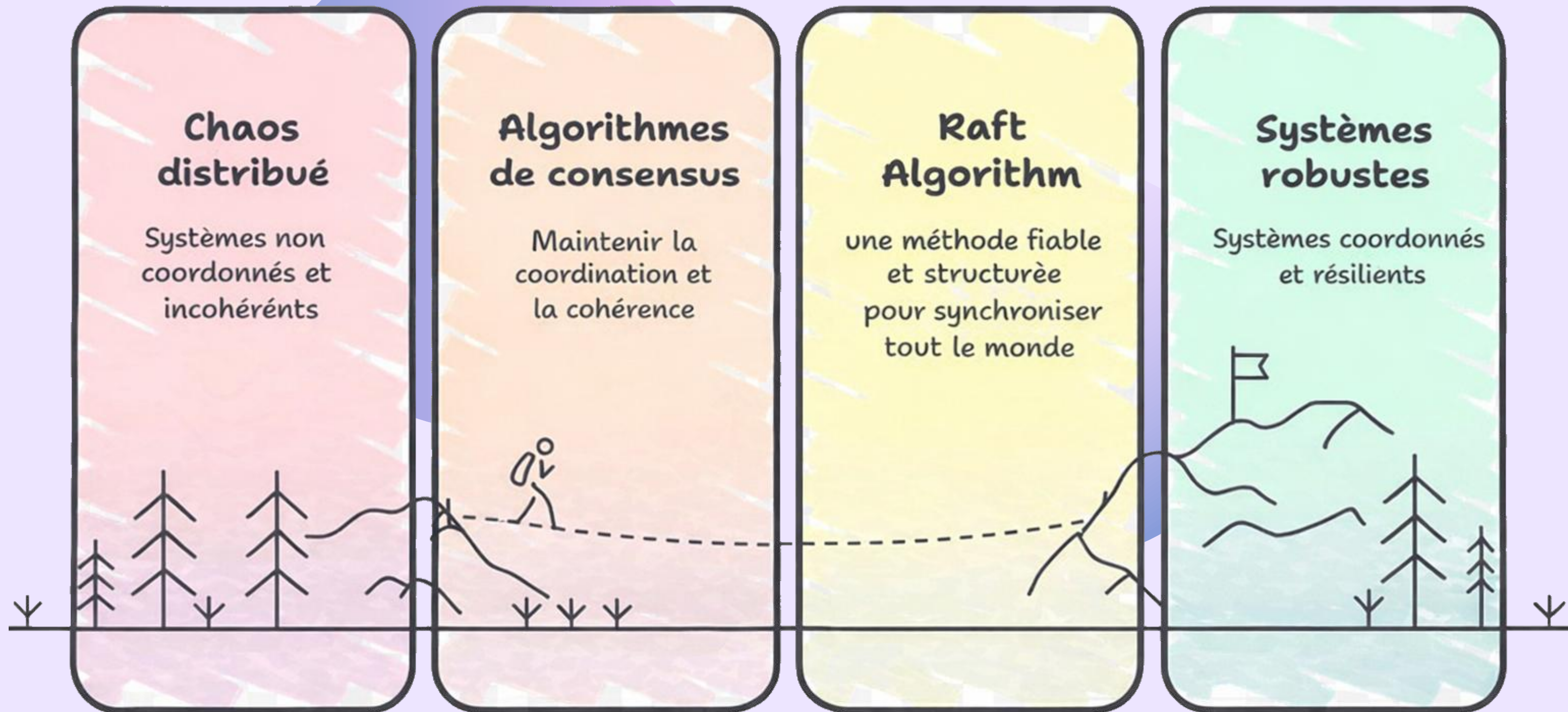
● Maintien de la disponibilité

Pendant la transition, le cluster fonctionne avec l'ancienne et la nouvelle configuration simultanément. Une fois la majorité des deux groupes d'accord, la nouvelle configuration devient effective, garantissant cohérence et disponibilité.



Exemple scénario Raft





Atteindre le consensus dans les systèmes distribués

Merci !

Des questions ?

Les sources

● Articles

LinkedIn

<https://www.linkedin.com/advice/0/what-raft-consensus-algorithm-how-does-work-skills-blockchain-p4rmf?lang=fr>

Wikipédia

- https://fr.wikipedia.org/wiki/Probl%C3%A8me_du_consensus
- [https://en-wikipedia-org.translate.goog/wiki/Raft_\(algorithm\)?_x_tr_sl=en&_x_tr_tl=fr&_x_tr_hl=fr&_x_tr_pto=rq](https://en-wikipedia-org.translate.goog/wiki/Raft_(algorithm)?_x_tr_sl=en&_x_tr_tl=fr&_x_tr_hl=fr&_x_tr_pto=rq)

GitHub

<https://raft.github.io/>

IBM

<https://www.ibm.com/fr-fr/think/topics/etcd>

● Images & designs

Fait par Guillemette sur Canvas

<https://www.canva.com>

Licence Education

Image “Atteindre le consensus dans les systems distribués” original :

<https://aws.plainenglish.io/raft-consensus-algorithm-explained-making-distributed-systems-understandable-de032f13b372>